Project 3: SSD Performance Profiling

Samson Kempiak
ECSE 4320/6320 Advanced Computer Systems
kempis@rpi.edu

October 22, 2026

Abstract

We profile a Micron 2210 512 GB QLC NVMe SSD on a PCIe 3.0 ×4 link with `O_DIRECT` I/O against disposable files, using a `libaio` benchmark cross-checked against FIO, in one scripted run that starts on a trimmed, idle drive and idles it between experiments. At QD=1, 4 KiB random reads take 50.3 µs (19 863 IOPS) and writes 15.3 µs, because the write buffer hides the flash program. With one request in flight, sequential reads reach only 1 847 MiB/s even at 1 MiB; at QD=32 they peak at 2 102 MiB/s, 56 % of the 3756 MiB/s link, so the NAND back end is the ceiling. Random-read IOPS saturate at QD 64, and Little's Law confirms that the harness keeps the queue full. The SLC cache absorbed 20.8 GiB at 1 061 MiB/s before sequential writes fell to 63 MiB/s, but its size depends on drive state (3.8 and 3.1 GiB in two earlier runs), and after a full fill it did not recover within a minute. Read p99.9 reaches 6.4× the median at QD=128, and randomly written data reads 6–9× slower sequentially. Dropping `O_DIRECT` over-reports throughput 19× because it measures the page cache.

1 Introduction

NVMe SSDs deliver high IOPS and throughput only when requests are issued concurrently. Like memory, storage shows a queueing-driven throughput–latency tradeoff: deeper queues raise utilization up to a saturation knee, and past it latency climbs for little extra throughput. We characterize this on a Micron 2210 512 GB QLC NVMe SSD, using `O_DIRECT` I/O against disposable files, a small `libaio` benchmark as the primary engine and FIO as a cross-check (Sections 2 and 3). The report answers:

1. What are the QD=1 latencies for small random and large sequential I/O? (Section 4)
2. How do block size, access pattern, read/write mix and queue depth shape IOPS, MiB/s and latency, and where is the PCIe interface ceiling? (Sections 5 and 6)
3. Where is the queue-depth knee, by what criterion, and what does Little's Law say about the harness? (Section 7)
4. What do tail latency and device state reveal: working-set size, SLC-cache exhaustion and steady state, data layout, and temperature? (Sections 8 and 9)
5. Which setup mistake produces misleading numbers? (Section 10)

Table 1: Measurement environment.

CPU	Intel(R) Core(TM) i9-10900F CPU @ 2.80GHz (x86_64)
SIMD ISA	sse sse2 ssse3 sse4_1 sse4_2 avx avx2 fma f16c – widest 256-bit (ymm)
Topology	1 socket(s), 10 cores/socket, 1 threads/core (10 of 20 logical CPUs online); NUMA: 1 node(s)
SMT	off
Caches	L1d 320 KiB (10 instances); L2 2.5 MiB (10 instances); L3 20 MiB (1 instance)
Memory	63 GiB total; DIMMs: 2 × 32 GiB DDR4-3200, one per channel (A, B), running at 2933 MT/s (dmidecode)
Pages	base 4096 bytes; THP: always
OS / kernel	Arch Linux; 7.2.9-arch1-1
Frequency policy	governor: performance; driver: intel_pstate; turbo: disabled; EPP: performance
Pinning / isolation	a single I/O-issuing thread, not CPU-pinned (storage I/O is device-bound, not CPU-bound, below the host-saturation queue depths of Section 7); no isolcpus
Compilers	gcc (GCC) 16.2.1 20260810; clang version 23.1.3
Tools	perf: perf version 7.2.11-1 (perf_event_paranoid 0); FIO: fio-3.42; MLC: present
GPU (optional)	NVIDIA GeForce RTX 3070; CUDA not available (no CUDA)
SSD	Micron 2210 `MTFDHBA512QFD`, 512 GB QLC NVMe [3], firmware `P6MA001` (SMART log taken with root after the run, Section 11)
Interface	PCIe 8.0 GT/s ×4 (Gen3), current and maximum, from sysfs. Raw ceiling 8 GT/s × 4 × $\frac{128}{130}$ = 3.94 GB/s (3756 MiB/s; 128b/130b coding [4]), roughly 3.2 GB/s to 3.5 GB/s after TLP/DLLP overhead
Block layer	scheduler `none`, `nr_requests` 1023, 16 I/O queues, volatile write cache "write back"
Target	disposable files on the SSD's ext4 `/` partition (`nvme0n1p3`, `relatime`, no online discard) via `ACS_SCRATCH`: a 2 GiB test file for the sweeps and a 60 GiB file for the device-state runs
Temperature	NVMe composite sensor (`hwmon*/temp1_input`), logged per repetition and every second in the sustained runs
Env. snapshot	2026-10-21 20:46 EDT (each raw run directory also stores its own `env.txt`)

2 Methodology and Safety

2.1 Platform

The machine also has a SATA HDD (`/dev/sda`); each run's `env.txt` resolves the scratch partition to its parent disk and records its `rotational` flag, model, firmware, link and temperature, so a run cannot be silently mislabelled (Section 12 describes how this check once failed).

2.2 Data-loss safeguards

- **Targets.** Only disposable files in a dedicated scratch directory are written, and both are deleted when `run.sh` exits, including on interrupt.
- **Gate.** `scripts/safety.sh` runs before every experiment and refuses block or character devices (also via symlinks), mountpoints, directories, paths outside the scratch directory and targets without enough free space; the scratch directory itself may not be `/`, `\$HOME`, a filesystem root or foreign-owned. It only `stat`s paths (Section 3).
- **Benchmark.** `blkbench` `lstat`s its target, refuses anything but a regular file and opens with `O_NOFOLLOW`; no raw device is ever opened.
- **Space.** The large device-state file is refused unless 10 GiB stay free.

2.3 I/O engines and configuration

The primary engine, `build/blkbench`, is a small O_DIRECT `libaio` benchmark (the engine FIO also uses [1]). It keeps exactly QD requests in flight in a closed loop and records every I/O's submit-to-completion latency. FIO 3.42 runs the same points through `fio/job.fio` (Section 3), and `ENGINE=fio` routes every sweep through FIO.

Engine	`libaio`; `O_DIRECT`; 4 KiB-aligned buffers and offsets; 64-bit uniform random offsets (splitmix64)
Timing	4 s per repetition (6 s for tail); write points also stop after 256 MiB; preconditioning absorbs warm-up
Repetitions	5 per point (3 for block-size writes; 3 per pass, 6 in total, for the working set); median with inter-quartile-range (IQR) bars
Data pattern	incompressible xorshift64* buffers, fixed per queue slot (we did not test whether the 2210 deduplicates)
Latency metric	per-I/O submit→completion; p50/p95/p99/p99.9 from the full sorted sample
Preconditioning	sequential O_DIRECT write over the test file at the start of every experiment and before every sequential-read group (Section 9.3)
Driver	`SSD_Performance_Profiling/scripts/run.sh`; `src/blkbench.c`

2.4 Execution, run order and idle periods

All results except `pitfall` and `layout` come from one scripted run, `scripts/rerun_clean.sh` (run `20261022-021116`, 2026-10-22, 02:11–04:08 EDT; Table 2), started by the author in a terminal. Beforehand we set the governor to `performance`, disabled turbo, SMT and ACPI GPE 0x42, ran `sudo fstrim -v /` (42.8 GiB trimmed, about 01:41) and left the machine idle for 30 min. The script then checked free space (77 GiB), the TRIM in the system journal and the drive itself: 0 MiB read or written and 0 % busy over 10 s.

The run puts the large sequential fill first, on the clean drive, and gives every later experiment an idle period: 20 min before `working_set`, 10 min before `tail` and 5 min before each other step. Write points also wait before they start: 120 s in `zero_queue` and `rw_mix`, 15 s in `block_size` and `queue_depth`. `pitfall` and `layout` were not repeated; they come from our first run (`20261021-103201`).

Idle period. An *idle period* means that the drive received no I/O from our scripts for a stated time (5 min to 30 min), so that the controller could finish background SLC-to-QLC folding and garbage collection; an *idle drive* is one measured after such a period.

Superseded runs. Two earlier runs are not used for the results. The first (`run.sh all`, then `SUSTAIN_GIB=60 run.sh device_state`; runs `20261021-103201` and `20261021-105439`, 2026-10-21) ran all experiments back to back without TRIM or idle periods, so each inherited the SLC-cache fill and background work of the one before. A partial "lean" re-run that evening (`20261021-220614`) re-measured only the most affected points, with a short fill, and could only be merged with the first. Because of these run-order confounds both were replaced by the run above; their raw data stay in the repository, and we cite them only where they show how drive state changes a result.

3 Validation and Sanity Checks

`make -C SSD_Performance_Profiling test` (`scripts/validate.sh`) checks that the benchmark measures the SSD and not an artifact. It writes one 64 MiB scratch file and deletes it.

Table 2: The reported run (`rerun_clean.sh`, 2026-10-22) in execution order: wall-clock time of each step from the run log, including its idle period, and the most it could write (byte caps plus preconditioning). `pitfall` and `layout` are from the first run.

Step	Time	Max written	Notes
`sustained`	16.2 min	72 GiB	60 GiB fill + ≤8 GiB random + 60 s idle + ≤4 GiB recovery
`working_set`	24.1 min	6 GiB	20 min idle; on the 60 GiB file, which it then deletes
`tail`	12.6 min	2 GiB	10 min idle; fresh 2 GiB precondition; reads only
`zero_queue`	9.9 min	7 GiB	5 min idle; 120 s before each write point
`rw_mix`	11.6 min	4 GiB	5 min idle; 120 s before each mix with writes
`block_size`	22.6 min	24 GiB	5 min idle; 56 points; 15 s before each write point
`queue_depth`	11.0 min	14 GiB	5 min idle; reads and writes, QD 1–256
`fio_check`	9.0 min	10 GiB	5 min idle; 7 points, both engines, order alternating
`pitfall`	0.8 min	2 GiB	first run; reads only (plus re-fill)
`layout`	2.2 min	6 GiB	first run; includes a 4 GiB random overwrite

- **Safety gate.** `safety.sh` refuses both block devices, a symlink to `/dev/sda` inside the scratch directory, a directory, a path outside it (`/etc/passwd`), a mountpoint (`/`), and scratch directories at `/`, `\$HOME`, a filesystem root (`/dev/shm`) or not owned by the user (`/usr/share`); each test checks that the refusal names the intended rule. `blkbench` independently refuses a directory and a symlink.
- **Incompressible data.** The preconditioned file does not shrink under `gzip -1`.
- **Physical orderings.** 128 KiB sequential beats 4 KiB random throughput, and QD 1→8 raises $L = \lambda W$ without exceeding QD (Section 7).
- **Direct I/O is real.** The `-buffered` path reaches 19× the sequential and 67× the random `O_DIRECT` IOPS (Table 10), so the real runs bypass the page cache; `make test` asserts a gap of more than 5×.
- **Interface ceiling and identity.** No result may exceed the 3756 MiB/s link (the highest, 2 102 MiB/s, does not), and every `env.txt` records the parent disk, `rotational`=0, model, firmware and link.

We did not collect `iostat` or `perf` data, so effective queue depth is checked in-band (Table 7).

3.1 Cross-check against FIO

`run.sh fio_check` runs seven points (QD=1 and 32; reads and writes; 4 KiB random and 128 KiB sequential) through both engines, alternating them repetition by repetition. FIO's `lat` percentiles measure the same submit→completion time (log-bucketed, ∼3 % bin error). We treat agreement within 10 % in IOPS and p50 as validation (Table 3).

Reads agree: IOPS ratios of 0.980–1.000 and p50 within 2 %. Every step preconditions its test file first, so the 128 KiB sequential points read freshly written data at the same rate as Sections 4 and 5 (1 489 MiB/s at QD=1).

Writes vary more, mostly with drive state. The write points give ratios of 0.983 (sequential), 0.930 and 1.007 (random, QD=1 and 32), with p50 within 2 µs. In Table 4, position within a pair moves the sequential write from 8 660 to 6 433 IOPS for the same engine, more than the engine gap, and the QD=32 random-write ratio was 0.78 in our superseded first run, which a fixed engine bias

Table 3: `blkbench` vs. FIO on identical points (medians of 5 interleaved repetitions per engine; latencies in µs).

Pattern	Block	QD	IOPS		FIO/blk	p50		p99	
			blk	FIO		blk	FIO	blk	FIO
rand read	4 KiB	1	19 962	19 553	0.980	49.8	50.4	51.9	52.0
rand read	4 KiB	32	135 728	135 771	1.000	212.0	216.1	547.9	544.8
seq read	128 KiB	1	11 912	11 883	0.998	81.3	80.4	196.8	197.6
seq read	128 KiB	32	16 350	16 349	1.000	1 851.8	1 859.6	3 061.8	3 063.8
rand write	4 KiB	1	65 284	60 738	0.930	14.7	15.4	16.8	17.8
rand write	4 KiB	32	240 199	241 830	1.007	120.1	119.3	147.3	154.6
seq write	128 KiB	1	8 606	8 463	0.983	99.5	97.8	302.7	309.2

would not allow. FIO's slower QD=1 random write (0.93 in both runs) may reflect a small per-I/O host cost. We give write numbers from either engine a ∼15 % uncertainty.

Table 4: Write points of `fio_check` split by engine and by position within each pair of repetitions: median IOPS of 3 (`blkbench` first, FIO second) or 2 (the other two columns) repetitions per cell.

Point	blkbench 1st	blkbench 2nd	FIO 1st	FIO 2nd
4 KiB rand, QD=1	65 594	65 034	62 072	60 569
4 KiB rand, QD=32	244 075	239 060	249 188	241 830
128 KiB seq, QD=1	8 660	6 433	7 969	8 498

4 Zero-Queue Baselines

Hypothesis (from first principles, then compared with the data)

At QD=1 a single I/O is in flight, so latency is the full host-stack + controller + flash path. We expect 4 KiB random reads in the tens of microseconds (flash read dominates) and 128 KiB sequential transfers to be bandwidth-limited at a few hundred µs. Writes should be *faster* than reads at QD=1 because the SSD's DRAM/SLC write buffer acknowledges them before the flash program completes.

Fixed	iodepth 1; one job; `O_DIRECT`; 5 repetitions of 4 s (write points also stop at 256 MiB and start after a 120 s pause)
Cases	4 KiB random read, 4 KiB random write, 128 KiB sequential read, 128 KiB sequential write
Order	random read → random write → sequential re-fill of the file → sequential read → sequential write (the re-fill keeps the sequential read independent of the random writes; Section 9.3)
Script	`SSD_Performance_Profiling/scripts/run.sh zero_queue`

4.1 Analysis

The QD=1 numbers identify an NVMe SSD, not a rotational disk: 4 KiB random reads take 50.3 µs, dominated by the NAND read plus the NVMe/`libaio` host path, with no seek. Random *writes* take only 15.3 µs (65 073 IOPS), because the controller acknowledges them from its write buffer (SLC cache plus SRAM) before the flash program finishes; a read has no such buffer. The 128 KiB

Table 5: QD=1 performance on the Micron 2210 NVMe SSD: medians over repetitions, with the IQR in brackets for IOPS and p99; latencies in µs. Random reads pay the full flash-read latency; random writes are faster because the write buffer absorbs them.

Pattern	Block	IOPS [IQR]	MiB/s	Mean	p50	p95	p99 [IQR]	p99.9
rand read	4 KiB	19 863 [19 853, 19 894]	77.6	50.3	50.1	50.9	52 [52, 52]	89
rand write	4 KiB	65 073 [64 872, 65 114]	254.2	15.3	14.8	16.0	17 [17, 17]	77
seq read	128 KiB	11 908 [11 906, 11 912]	1 488.6	83.9	81.2	114.0	197 [197, 197]	481
seq write	128 KiB	8 701 [8 661, 8 746]	1 087.6	114.9	99.6	200.8	252 [250, 252]	349

sequential read takes 84 µs per request (1 489 MiB/s): one outstanding request leaves the device idle between transfers, which Sections 5 and 7 remove. The p99.9 column already shows background flush and GC outliers (Section 8).

The 128 KiB sequential write runs at 1 088 MiB/s, the SLC ingest rate of Section 5, and is stable (IOPS IQR 8 661–8 746). In our first run, where it followed the 2 GiB re-fill without a pause, two of five repetitions stalled below 900 IOPS; after the 120 s pause none did. The re-fill itself exists because a discarded pilot that read sequentially right after the random writes got only a fraction of the sequential-read rate (Section 9.3).

5 Block-Size Sweep

Hypothesis (from first principles, then compared with the data)

As the block grows from 4 KiB to 256 KiB at QD=1, small-block I/O should be *IOPS-limited* (throughput = IOPS × block, rising with block size) and large-block I/O should approach a *throughput ceiling* where IOPS falls as 1/block. Sequential should beat random at every block size, with the gap largest at small blocks where per-op overhead dominates.

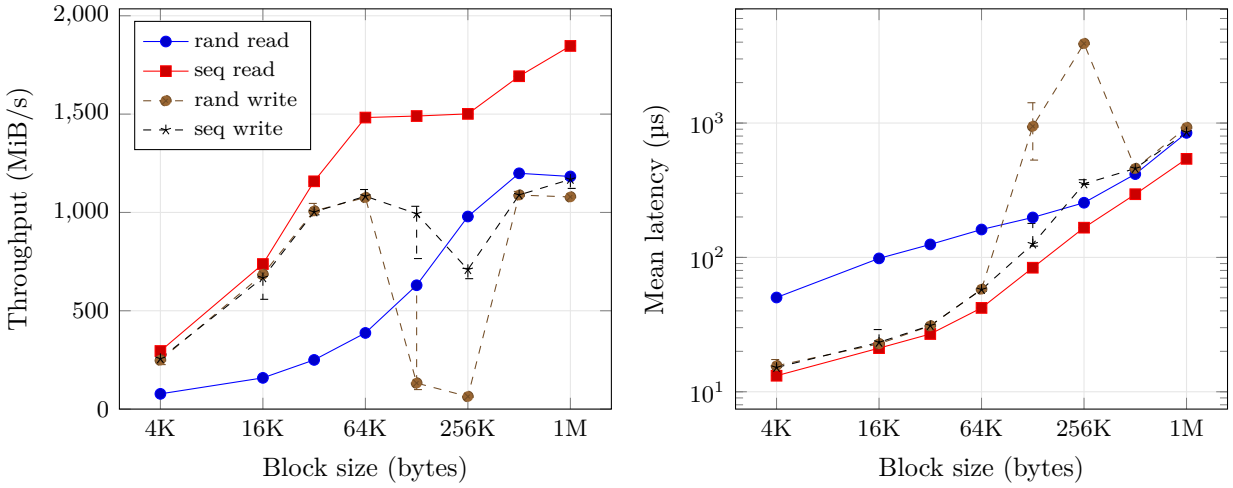
Fixed	one job; `O_DIRECT`; 5 repetitions of 4 s for reads; 3 repetitions capped at 256 MiB each for writes, each write point after a 15 s pause
Varied	block size 4 KiB–1 MiB (8 sizes); random vs. sequential; 100% read vs. 100% write; QD=1 (all four combinations) and QD=32 (sequential read and write, random read)
Ceiling	PCIe 3.0 ×4: 3756 MiB/s raw (Section 2)
Script	`SSD_Performance_Profiling/scripts/run.sh block_size`

5.1 Analysis

Random reads at QD=1 are latency-bound, not size-bound. From 4 KiB to 256 KiB, IOPS fall only from 19 851 to 3 917, so throughput climbs from 78 to 979 MiB/s while latency grows only 5.1× for a 64× larger block. We read this as $W(b) \approx W_0 + t_{\text{xfer}}(b)$: a fixed cost $W_0 \approx 50$ µs (NAND sense plus command overhead) plus a transfer term that grows sub-linearly while the controller stripes a request across dies. Beyond 256 KiB latency doubles with size and throughput plateaus at 1 199–1 183 MiB/s.

Sequential reads at QD=1 hit a single-request limit. Read-ahead cuts the 4 KiB sequential read to 13.2 µs (random: 50.3). From 64 to 256 KiB throughput is flat at 1 482–1 501 MiB/s, reaching 1 847 MiB/s at 1 MiB, only 49 % of the link: the limit is how fast the controller streams one command.

Deep queues do not reach the link ceiling (Figure 2). At QD=32 sequential reads flatten at 2 102–2 076 MiB/s (1 937 MiB/s at 1 MiB), about 56 % of the raw link, well below the 85 % to



(a) QD=1 throughput.

(b) QD=1 mean latency (same runs).

Figure 1: QD=1 block-size sweep, reads (solid) and writes (dashed); medians with IQR bars. Random-read throughput rises with block size because latency grows more slowly than the transfer size. Sequential reads stay near 1.45 GiB/s from 64 to 256 KiB and reach 1.8 GiB/s at 1 MiB, a single-request limit well below the 3756 MiB/s link. Writes stay at the ∼1.05 GiB/s SLC ingest rate from 64 KiB up, except at 128 KiB and 256 KiB, where the random writes (and, less, the sequential ones) met a cache-exhaustion episode (see text).

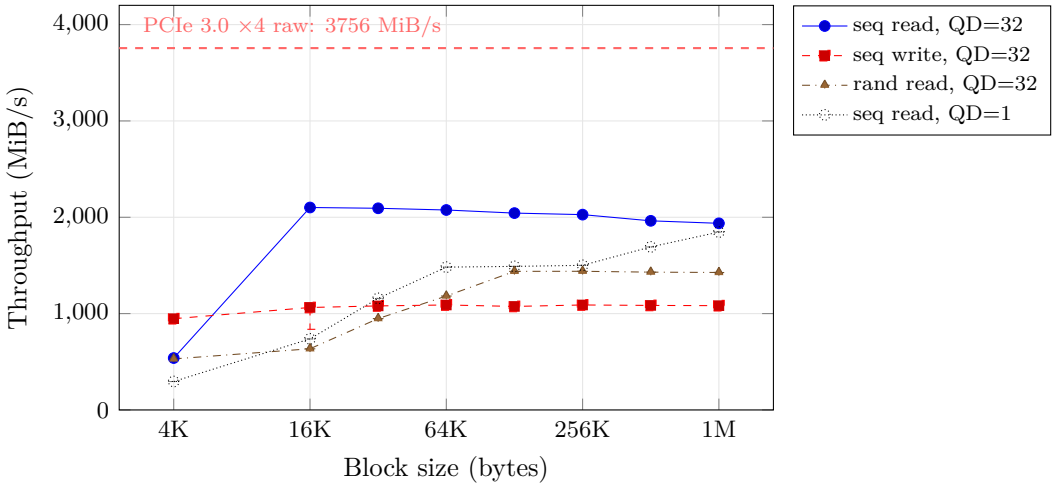


Figure 2: Sequential throughput at QD=32 against the measured-link ceiling (red), with the QD=1 read curve for reference. Deep queues remove the per-request latency limit, so the remaining gap to the line is the device's internal bandwidth plus protocol overhead.

90 % protocol overhead allows. The NAND/controller back end is the limit, consistent with Micron's 2200 MB/s read and 1070 MB/s write rating (≈2098 and 1020 MiB/s) [3]. QD=32 sequential writes are flat at 1 090 MiB/s, the SLC ingest rate (Section 9).

Random reads at QD=32 reach 135 601 IOPS at 4 KiB, about the sequential rate at that size (137 941), plateau at 1 439 MiB/s from 128 KiB up and give 1 427 MiB/s at 1 MiB, against 1 937 for sequential: random placement costs about a quarter of the bandwidth even with 32 large requests in flight.

Writes at QD=1 post 63 754 IOPS at 4 KiB (reads: 19 851), and random and sequential curves coincide (1 076 vs. 1 082 MiB/s at 64 KiB), flattening at the same ∼1.05 GiB/s ingest rate as at QD=32: the buffer acknowledges before the NAND program, so writes reach their ceiling with one request in flight.

A drive-state episode, not a block-size effect. The QD=1 random writes ran from small to large blocks. During the 128 KiB point, after about 3.3 GiB of random writes in this step (plus its 2 GiB preconditioning pass), the rate fell from 1087 to 132 and then 66 MiB/s, and all three 256 KiB repetitions ran at 61 MiB/s to 66 MiB/s with p99 of 34 ms to 40 ms: the direct-to-QLC rate of Section 9. After the next 15 s pause the 512 KiB and 1 MiB points were back at 1 088 and 1 079 MiB/s. The sequential writes, measured later, dipped less (709 MiB/s at 256 KiB). Because larger blocks recovered, we read the collapse as the SLC cache running out of free blocks, which the 60 GiB fill at the start of the run had left with little headroom (Section 9), not as an effect of block size. Our first run showed the same collapse from 256 KiB to 1 MiB.

6 Read/Write Mix

Hypothesis (from first principles, then compared with the data)

At a fixed block size and queue depth, a mixed read/write workload should deliver *fewer* total IOPS than either pure workload, not a weighted average: reads and writes contend for the controller and flash channels, and a read queued behind a slow program/erase inherits its latency. We expect pure writes to post the highest IOPS at QD=16 (write-buffer absorption) and the 50/50 mix to be the worst.

Fixed	4 KiB random; iodepth 16; one job; `O_DIRECT`; 5 repetitions of 4 s (points with writes also stop at 256 MiB and start after a 120 s pause)
Varied	reads per 10 I/Os, deterministically interleaved: 10 (100% R), 7 (70/30), 5 (50/50), 0 (100% W)
Script	`SSD_Performance_Profiling/scripts/run.sh rw_mix`

Table 6: Read/write mix at QD=16, 4 KiB random: medians with the IQR in brackets; latencies in µs.

Mix	Total IOPS	Mean latency	p99
100% read	107 086 [106 814, 107 141]	147.7 [147.7, 148.1]	303 [303, 303]
70/30	86 025 [83 353, 86 957]	184.0 [182.1, 190.0]	667 [664, 680]
50/50	74 758 [74 694, 74 759]	212.5 [212.5, 212.9]	984 [894, 1 026]
100% write	254 872 [253 390, 256 577]	62.5 [62.0, 62.9]	71 [69, 74]

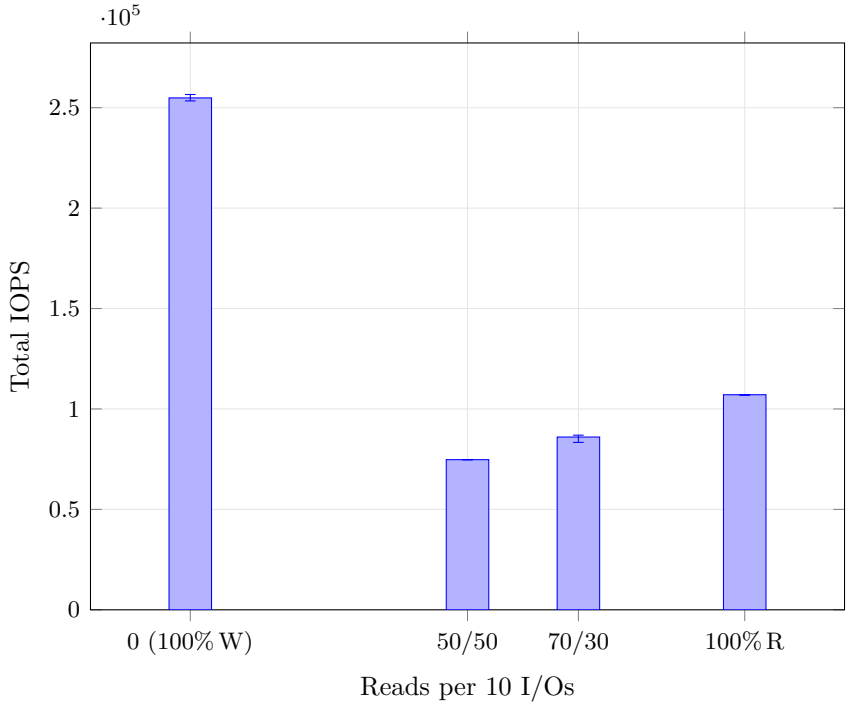


Figure 3: Total 4 KiB random IOPS at QD=16 across read/write mixes (median, IQR bars). Both pure workloads beat the mixes; the 50/50 mix is the worst.

6.1 Analysis

Pure writes are fastest, at 254 872 IOPS and 63 µs mean, because the controller absorbs them into its SLC/SRAM buffer; pure reads reach 107 086 IOPS. Mixing is worse than either: 70/30 drops to 86 025 IOPS and 50/50 to 74 758, below even pure reads. A write occupies a die (program, or the buffer flush it triggers), reads mapped to that die queue behind it, and interleaving breaks up the buffer's batching.

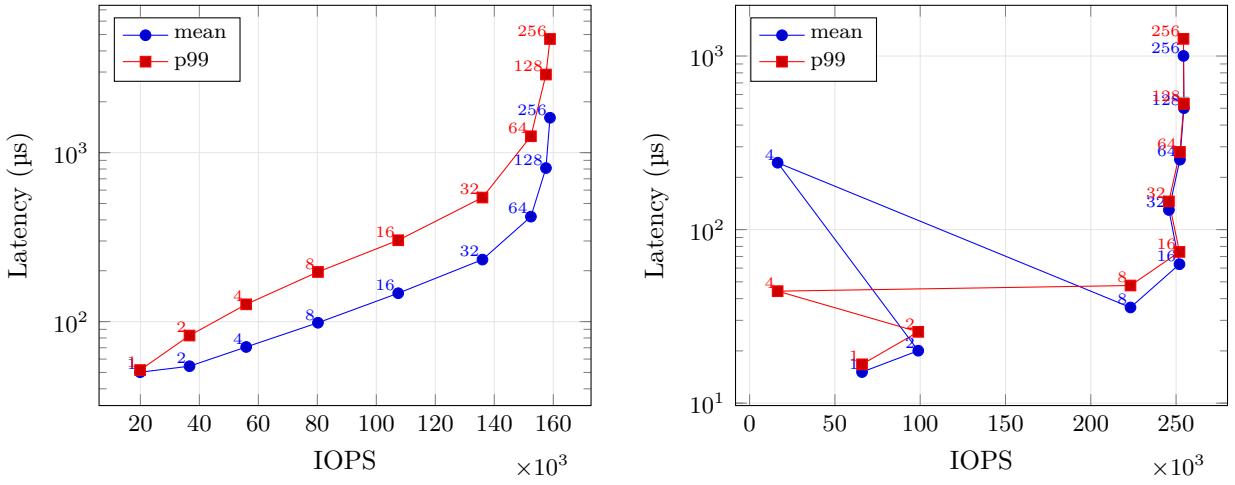
p99 rises from 303 µs for pure reads to 667 (70/30) and 984 µs (50/50), while pure writes have the lowest p99, 71 µs. In the 50/50 mix the median falls to 57 µs, because half the requests are buffered writes, while its p99 is more than 3× that of pure reads, plausibly reads waiting behind writes. A "70% read" workload is thus not a weighted average of the pure results; mixes must be measured, not interpolated.

7 Queue-Depth Sweep

Hypothesis (from first principles, then compared with the data)

IOPS should rise steeply with queue depth as the SSD's parallel NAND channels fill, then saturate at a knee beyond which latency grows roughly linearly with queue depth for no extra throughput. By Little's Law the number of outstanding I/Os $L = \lambda W$ should track the configured iodepth while the queue stays full; where it stops tracking reveals a host-side or engine limit.

Fixed	4 KiB random; one job; `O_DIRECT`; 5 repetitions of 4 s (writes also stop at 256 MiB and start after a 15 s pause)
Varied	iodepth 1, 2, 4, …, 256; 100% read and 100% write
Knee rule	the smallest QD whose doubling raises median IOPS by less than 10 % (computed by `run.sh` into `queue_depth_knee.csv`)
Effective QD	checked in-band with Little's Law (Table 7); no `iostat` data
Script	`SSD_Performance_Profiling/scripts/run.sh queue_depth`



(a) 4 KiB random read.

(b) 4 KiB random write.

Figure 4: Throughput vs. latency as queue depth grows; each point is labelled with its QD. Bars show the IQR across repetitions in both IOPS and latency. Read IOPS reach 152 416 at the knee (QD 64) and at most 158 887 at QD=256; deeper queues then add only latency. In (b), the QD=4 point at 16 430 IOPS is a cache-exhaustion stall (see text).

7.1 Knee

We define the knee as *the smallest QD whose doubling raises median IOPS by less than* 10 %: past it, most extra concurrency becomes latency rather than throughput.

Reads. The rule gives QD 64 (152 416 IOPS, 418 µs mean). Doubling to QD 128 adds only 3.3 % IOPS, while QD 32→64 still gained 12 %, so the knee lies between 32 and 64.

Writes. The rule returns QD 2 (98 866 IOPS), because the next point collapsed: at QD=4 IOPS change by −83.4 %, to 16 430. All five QD=4 repetitions ran at 61 MiB/s to 67 MiB/s, the post-cliff QLC rate of Section 9. Their median latency stayed at 16.8 µs and p99 at 44 µs, but under 1 % of writes waited ∼33 ms (p99.9) for the buffer to drain, which sets the mean of 243 µs. L/QD stays near 1 (Table 7), so the queue was full and the drive simply accepted writes no faster. The point came after 2.5 GiB of writes in this step and 24 GiB in `block_size`, with the cache not fully recovered from the 60 GiB fill; the first QD=8 repetition was still slow (20 076 IOPS), the other four ran at 223k. Ignoring the stalled point, IOPS rise from 65 911 at QD=1 to 251 993 at QD=16, the 10 % rule would place the knee at QD=16 (QD 8→16 gains 13 %, 16→32 −2.5 %), and the curve stays at 245 788–254 579 up to QD=256. The rule's QD=2 is thus an artifact of the stall; in-cache writes saturate at QD ≈ 16. The plateau of ∼250k IOPS × 4 KiB ≈ 0.95 GiB/s is the SLC ingest rate: writes saturate the buffer's intake, not the NAND parallelism that limits reads.

7.2 Little's Law as a consistency check

Little's Law [2] states $L = \lambda W$ for the mean number in system L, throughput λ and mean time in system W. In a closed loop it holds by construction: $\lambda W = \mathrm{QD} \cdot W/(W + g)$, where g is the host gap between reaping and resubmitting. So $L/\mathrm{QD} \approx 1$ shows that the timers are consistent and the host keeps up ($g \ll W$), while a clearly lower ratio would expose a host bottleneck. As a control, `make test` on tmpfs, where I/O completes inside `io_submit`, measured $L/\mathrm{QD} \approx 0.57$ ($L = 4.56$ at QD=8; `data/raw/validate_tmpfs/20261021-231433.txt`), which the harness flags. L includes time in the kernel block layer, so device-queue occupancy can only be lower.

Table 7: Little's Law per queue depth: λ (IOPS) and W (mean latency, µs) are medians with the IQR in brackets; $L = \lambda W$ is computed per repetition and then summarized. `4 KiB` random.

Mode	QD	λ (IOPS) [IQR]	W [IQR]	$L = \lambda W$ [IQR]	L/QD
read	1	19 906 [19 890, 19 907]	50.2 [50.2, 50.2]	1.00 [1.00, 1.00]	0.999
read	2	36 663 [36 638, 36 689]	54.4 [54.4, 54.5]	2.00 [2.00, 2.00]	0.998
read	4	55 881 [55 876, 55 887]	70.8 [70.8, 70.8]	3.95 [3.95, 3.95]	0.989
read	8	80 214 [80 206, 80 236]	98.4 [98.4, 98.5]	7.90 [7.90, 7.90]	0.987
read	16	107 426 [107 426, 107 465]	147.2 [147.2, 147.2]	15.82 [15.82, 15.82]	0.988
read	32	135 990 [135 942, 136 009]	232.8 [232.8, 232.9]	31.66 [31.66, 31.67]	0.99
read	64	152 416 [152 408, 152 422]	418.0 [418.0, 418.0]	63.71 [63.71, 63.71]	0.995
read	128	157 516 [157 466, 157 575]	810.8 [810.4, 811.1]	127.71 [127.71, 127.72]	0.998
read	256	158 887 [158 845, 158 902]	1 608.8 [1 608.8, 1 609.4]	255.64 [255.63, 255.64]	0.999
write	1	65 911 [65 542, 66 062]	15.1 [15.1, 15.2]	1.00 [1.00, 1.00]	0.996
write	2	98 866 [98 542, 99 415]	20.1 [19.9, 20.1]	1.98 [1.98, 1.98]	0.99
write	4	16 430 [15 784, 16 867]	242.8 [236.5, 252.7]	3.99 [3.99, 3.99]	0.997
write	8	223 280 [222 651, 223 495]	35.6 [35.5, 35.7]	7.95 [7.94, 7.95]	0.993
write	16	251 993 [250 142, 252 208]	63.2 [63.1, 63.7]	15.92 [15.91, 15.92]	0.995
write	32	245 788 [245 190, 246 561]	129.7 [129.4, 130.0]	31.88 [31.87, 31.88]	0.996
write	64	252 244 [250 471, 253 060]	253.0 [252.3, 255.0]	63.86 [63.84, 63.86]	0.998
write	128	254 579 [254 444, 256 116]	501.0 [498.1, 501.4]	127.57 [127.56, 127.58]	0.997
write	256	254 347 [253 494, 254 616]	1 001.5 [1 000.5, 1 006.4]	254.74 [254.73, 254.86]	0.995

Inference: internal parallelism. Applied to the device, with $\lambda_{\max} \approx 158\,887$ reads/s and an assumed service time S between 40 µs and the QD=1 latency (50 µs), only $L_{\mathrm{busy}} = \lambda_{\max} S \approx 6$–$8$ reads are in service at once; of the 64–256 outstanding past the knee, the rest wait in queues. With 1-Tbit dies (4 in a 512 GB drive; Micron does not publish this) that is about two reads per die, a plausibility argument only.

7.3 Analysis

Read IOPS climb from 19 906 at QD=1 to 107 426 at QD=16 and 135 990 at QD=32, then flatten at 152 416–158 887. Scaling is sub-linear from the start. Beyond the knee mean latency doubles with each doubling of QD (418→811→1 609 µs), the textbook saturation signature: with λ flat, every added request simply waits.

For reads L/QD stays at 0.987–0.999 up to QD=256 (Table 7), so the single `libaio` thread keeps the queue full and the saturation is the device's. For writes it is 0.990–0.998, including the stalled QD=4 point.

In practice, reads need QD ≈ 32–64 for nearly all the throughput at a quarter to half a millisecond; deeper queues only add latency. Writes need only QD ≈ 16 while the SLC cache has room; once it is full, QLC programming sets the rate at any depth (Section 9).

8 Tail Latency

Hypothesis (from first principles, then compared with the data)

Moving from a mid queue depth to the near-knee depth should raise the median modestly but stretch the upper percentiles much more: deeper queues mean more requests waiting behind the occasional slow flash operation, so the p99/p99.9 gap over p50 should widen.

Fixed	4 KiB random read; one job; 6 s runs, 5 repetitions; NVMe temperature recorded per repetition
Varied	iodepth 8 (mid), 32 and 64 (bracketing the knee of Section 7), 128 (past it)
Drive state	after `working_set` deleted the 60 GiB file, the drive was idle for 10 min; then a fresh 2 GiB test file was preconditioned sequentially (at only 69 MiB/s: the SLC cache had not yet recovered)
Script	`SSD_Performance_Profiling/scripts/run.sh tail` (inside `rerun_clean.sh`)

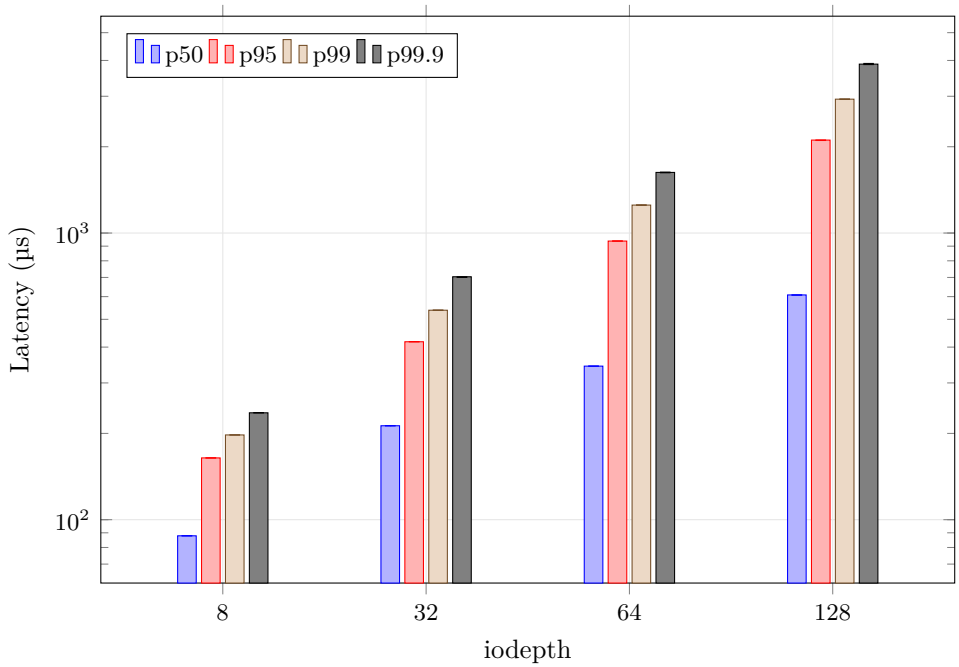


Figure 5: Latency percentiles at a mid queue depth (8), around the knee (32, 64) and past it (128); medians with IQR bars. Measured after a 10 min idle period on a freshly preconditioned file. The upper tail widens faster than the median as QD grows: p99.9/p50 rises from 2.7× at QD=8 to 6.4× at QD=128.

8.1 Analysis

After an idle period the tail widens with queue depth as hypothesised, but modestly. QD=8: p50 88, p99 198, p99.9 236 µs (p99.9/p50 = 2.7×); QD=32: 213, 538, 704 µs (3.3×); QD=64: p50 343, p99.9 1 627 µs (4.7×); QD=128: p50 608, p99.9 3 884 µs (6.4×). From QD=8 to 32 the median

grows 2.4× and p99.9 3.0×; from 32 to 64, 1.6× and 2.3×, while IOPS barely grow past the knee (151 879→156 859). Every added request waits, and one queued behind an occasional slow read waits longer, so the upper percentiles stretch more than the median; p99/p50 follows the same trend (2.2× to 4.8×). IOPS agree with the queue-depth sweep within 1 % at every depth (e.g. 135 623 vs. 135 990 at QD=32), so the read rates reproduce across experiments despite the slow preconditioning pass.

The tail depends on recent writes. In our superseded first run the tail experiment (`tail/20261021-103201`) started right after ∼11 GiB of random writes with no idle period. At QD=32 it delivered 64 475 IOPS, half the value here, at a median of 385 µs, and its p99.9/p50 rose from 4.1× at QD=8 to 24× at QD=128 (p99.9 16.0 ms). It was no hotter (50.9 °C to 52.9 °C vs. 42.9 °C to 54.9 °C here), and although the file and time of day also differed, the likely cause is reads waiting behind background folding and garbage collection. Queueing alone keeps the tail ratio at about 3–6×; recent heavy writes pushed it to 24×. For a latency SLA these percentiles matter more than the mean: at QD=32 one read in a thousand takes 704 µs after an idle period but took 2.6 ms after heavy writes, and past the knee these tails inflate fastest for almost no IOPS.

9 Device State and Working Set

Hypothesis (from first principles, then compared with the data)

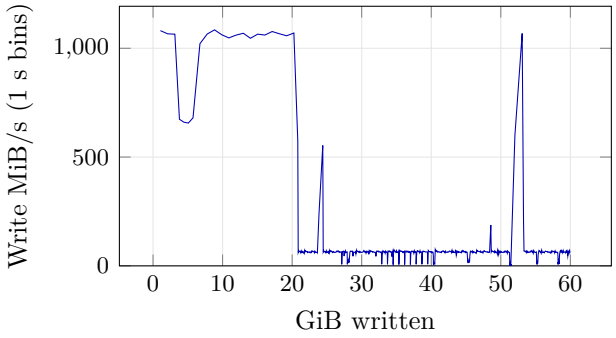
Back-to-back sustained sequential writes should first ride the SLC/DRAM write cache (a burst), then settle to a lower steady-state throughput once the cache must flush to QLC in the background. Occasional latency spikes from garbage collection should appear in the upper tail even while mean throughput is steady.

Hypothesis (from first principles, then compared with the data)

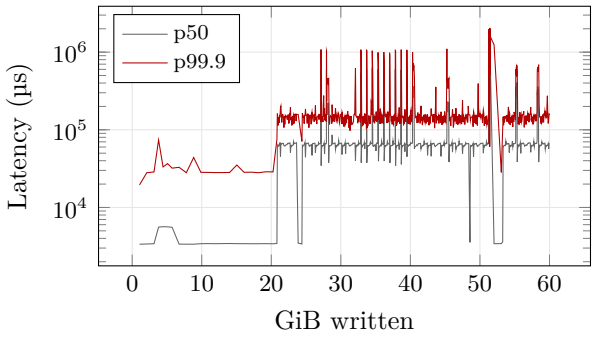
(Added before the large-file runs.) (i) A single sequential write far larger than the SLC cache should show a sharp *cliff*: burst throughput near the QD=32 write rate of Section 5, then a drop to the native QLC program rate, several times lower. (ii) After that, random 4 KiB writes reach a lower steady state than in the burst, with heavier tails from garbage collection. (iii) A short idle period lets the controller drain the cache and recover part of the burst rate. (iv) Random reads should slow as the working set grows from 1 GiB to 60 GiB, because more logical-to-physical map lookups miss the controller's mapping cache. (v) "Sequential" reads of data written randomly should be much slower than of data written sequentially. (vi) Temperature should rise during sustained writes but stay below the 86.85 °C warning threshold reported in sysfs (`temp1_max`).

`sustained`	sequential 128 KiB QD32 fill of a fresh 60 GiB file (time cap 3600 s); then 4 KiB QD32 random writes over the whole file (180 s or 8 GiB), a 60 s idle gap and a 128 KiB QD32 recovery probe (15 s or 4 GiB); logged every second
`working_set`	after a 20 min idle period, on the same 60 GiB file: 4 KiB random read (QD1, QD32) and write (QD32, 256 MiB cap) over its first 1, 4, 16 and 60 GiB, then in reverse order, 3 repetitions per pass
`layout`	first run (`20261021-103201`), 2 GiB file: 128 KiB sequential (QD1, QD32) and 4 KiB random reads after a sequential fill, then after a 4 GiB random overwrite (∼86 % of blocks)
Drive state	`fstrim` of 42.8 GiB at about 01:41, then 30 min idle; fill at 02:11. 77 GiB free on the scratch partition; other partitions (one untouched 351 GiB) not trimmed
Script	`rerun_clean.sh` (`sustained`, `working_set`); `run.sh layout`

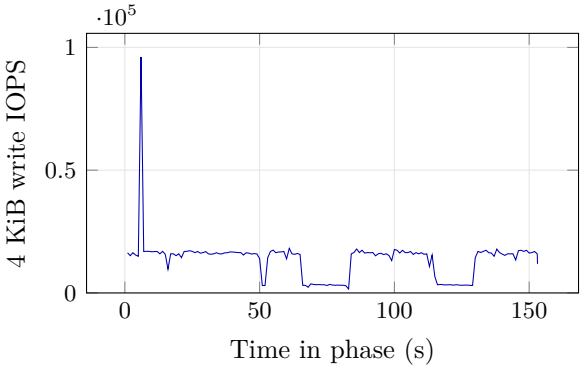
9.1 SLC-cache exhaustion and steady state



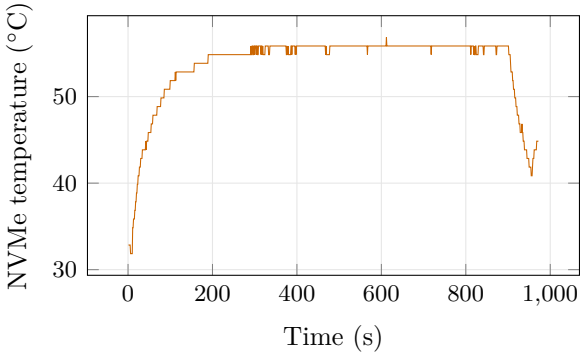
(a) Sequential fill: throughput vs. volume.



(b) Sequential fill: per-second latency.



(c) Random-write phase (QD32).



(d) Composite temperature, all phases.

Figure 6: Sustained-write experiment, one-second samples. (a) The SLC-cache cliff appears as the drop in throughput after 20.8 GiB; the fill then wrote the rest of the 60.0 GiB at the QLC rate. (b) The latency during the fill. (c) The random-write phase after the fill, which reached its 8 GiB cap after 153 s. (d) Temperature across the baseline idle, fill, random, idle and recovery phases.

Table 8: Sustained-write summary (computed by `run.sh` into `sustained_summary.csv`). Burst is the median of the leading seconds that stay above half the first second's rate. The cliff is the first second, starting a 5-second window, whose rate falls below half the burst; the table gives the volume written before it. Steady is the median of all seconds after the cliff. For the random phase the table gives the byte-weighted average, the per-second quartiles and the median per-second p99.

Burst sequential write (MiB/s) / duration (s)	1 061 / 22
Cliff after (GiB written / s)	20.8 / 22
Post-cliff steady sequential write (MiB/s)	63
Steady / burst	0.059
Fill: GiB, seconds, average MiB/s	60.0, 734, 84
Random 4 KiB write, QD32: phase average IOPS	13 698
per-second IOPS quartiles; median per-second p99 (µs)	13 236–16 472; 33 782
Recovery after 60 s idle: first / last 5 s (MiB/s)	68 / 62
NVMe temperature min–max (°C)	31.9–56.9

Analysis. The fill ran at the SLC ingest rate, 1 061 MiB/s, for 22 s and collapsed after 20.8 GiB, within two seconds, to 60 MiB/s to 69 MiB/s; the median per-second latency jumped from 3.4 to ∼63 ms. The post-cliff median, 63 MiB/s (0.059 of the burst, a 17× drop, far more than the "several times" of hypothesis (i)), held for the remaining 39 GiB with no downward drift: per-minute medians stayed at 62 MiB/s to 64 MiB/s except one minute, near 53 GiB, dominated by a slow episode. The rate was not smooth, though: 69 % of post-cliff seconds ran at 60 MiB/s to 70 MiB/s and 19 % below 20 MiB/s. The whole 60 GiB took 734 s (84 MiB/s on average). We take ∼63 MiB/s as the direct-to-QLC rate while the drive folds; hypothesis (i) holds. One detail we cannot explain: for four seconds, between 3.1 and 5.7 GiB written, the burst dipped to ∼670 MiB/s before full speed returned.

How large is the cache? It depends on drive state. After trimming 42.8 GiB and 30 min idle the cliff came at 20.8 GiB, close to the ∼20 GiB we expected. Two superseded runs found much less: 3.8 GiB in the lean run (`sustained/20261021-220614`, also after `fstrim` and ≥30 min idle, but the same day as a 60 GiB fill), and 3.1 GiB in the untrimmed first run (`sustained/20261021-105439`). A dynamic SLC cache sized by the free blocks the controller has available, common in QLC client drives [3], [6], would explain all three, but we did not measure the free-block count, so this remains an untested hypothesis.

The random-write phase (4 KiB, QD32, over the whole file) reached its 8 GiB cap after 153 s at 13 698 IOPS on average (per-second quartiles 13 236–16 472, median per-second p99 33.8 ms), about 18× below the in-cache rate (245 788 IOPS). Most seconds ran at ∼16k IOPS (∼63 MiB/s, the same QLC rate as the sequential fill), and in three episodes (34 of 154 seconds) at ∼3k IOPS. This supports hypothesis (ii): a much lower rate with heavy tails, sustained for 2.5 min.

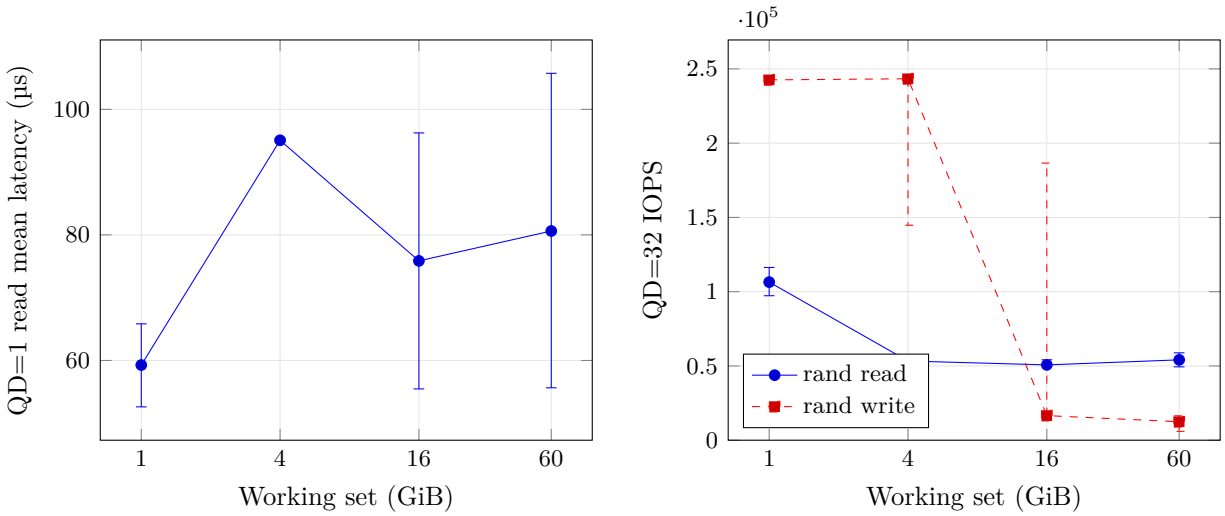
After the 60 s idle gap the recovery probe wrote at 68 MiB/s (last 5 s: 62 MiB/s), the QLC rate: no recovery, as in the first run (61 then 19 MiB/s). Hypothesis (iii) fails after a full 60 GiB fill. In the lean run, after a fill of only 4.3 GiB, the same gap restored the burst rate for at least 1 GiB (the probe's cap), so recovery depends on the fold backlog. Even the 20 min idle period before `working_set` freed only about 1.2 GiB of fast writes (Section 9.2). This is short-term behaviour, not the full-drive SNIA steady state [5].

Temperature. The composite temperature stayed at 31.9–56.9 °C here and 31.9 °C to 60.9 °C over the whole run, far below the 86.85 °C warning threshold, so hypothesis (vi) holds. The burst ran at up to 42.9 °C before the cliff, and the recovery probe still ran at the QLC rate at 41.9 °C to 44.9 °C, so the cliff is the cache, not thermal throttling.

9.2 Working set (LBA range)

Reads. QD=32 read IOPS halve between 1 and 4 GiB (106 463→53 341) in both passes and then stay flat (50 712 and 54 108 at 16 and 60 GiB). QD=1 latency also rises from 1 to 4 GiB (59.3→95.1 µs mean), but beyond that it depends on the pass more than on the span: 96 and 106 µs forward at 16 and 60 GiB, 55.5 µs at both in the reverse pass, hence the wide IQRs.

Hypothesis (iv) predicted a steady rise as more map lookups miss the controller's mapping cache. The data do not show it: nothing grows from 4 to 60 GiB, although the map for the span grows from 4 to 60 MiB (4 bytes per 4 KiB page). The step between 1 and 4 GiB did not appear in the lean run's 1–4 GiB spans on a freshly written 4 GiB file (flat QD=1 latency, QD=32 IOPS within 0.3 %), so it reflects this file's write history (a sequential fill mostly past the SLC cache, then 8 GiB of random overwrites) more than map size. Even at 4–60 GiB the file gives only about 40 % of the QD=32 read rate of the freshly written 2 GiB test file (135 990 IOPS), again pointing to where the data sit; why the first gigabyte is faster is untested. We count hypothesis (iv) as not supported.



(a) QD=1 random read latency.

(b) QD=32 random read and write IOPS.

Figure 7: 4 KiB random I/O restricted to the first 1, 4, 16 and 60 GiB of the 60 GiB file after a 20 min idle period (medians of 6 repetitions, 3 in each of a forward and a reverse pass, with IQR bars). QD=32 read IOPS halve between 1 and 4 GiB and then stay flat; beyond 4 GiB QD=1 latency differs between the passes (wide IQRs), and the write IQRs span a cache-exhaustion episode (see text).

Writes track time, not span. QD=32 writes ran at the in-cache rate (∼950 MiB/s) for the first ∼1.2 GiB (the 1 GiB span and the start of the 4 GiB one), fell to 61 MiB/s to 67 MiB/s at 16 and 60 GiB (as low as 19 MiB/s at the start of the reverse pass), and returned to ∼950 MiB/s from the second reverse 16 GiB repetition on. The medians in Figure 7b therefore show the cache state at each moment; the 20-minute idle period had freed only a little SLC space after the fill.

9.3 Data layout: why preconditioning order matters

Table 9: The same reads on the same 2 GiB file after a sequential fill and after a random 4 KiB overwrite (medians of 5 repetitions; first run, `20261021-103201`).

Read	after sequential fill	after random overwrite
128 KiB seq, QD=1 (MiB/s)	1 443	224
128 KiB seq, QD=32 (MiB/s)	2 093	229
4 KiB rand, QD=1 (IOPS)	18 803	18 747

An SSD places data in the order it was *written*, not by logical address, and Table 9 confirms hypothesis (v) strongly. After the random overwrite, QD=1 sequential reads drop from 1 443 to 224 MiB/s (6.4×), while 4 KiB random reads are unaffected. Our sub-prediction that QD=32 would hide the gap was *wrong*: it grows to 9.2×. Each fragmented 128 KiB request becomes up to 32 scattered page reads, and the drive completes ∼1830 such requests per second (about 58k page reads/s) at either depth, well below the ∼136k random reads/s it sustains at QD=32 (Section 7); why is untested. A sequential benchmark is meaningful only if the data's write history is controlled, which is why `run.sh` rewrites the file before every sequential-read group.

10 Misleading Setup

Hypothesis (from first principles, then compared with the data)

Dropping `O_DIRECT` lets the OS page cache serve repeated reads of the same region from RAM after the first pass, so a buffered benchmark will report throughput and latency that describe DRAM, not the SSD—orders of magnitude "better" than the real device, and entirely fictitious as a storage result.

Pitfall	Buffered I/O measuring the page cache instead of the device
Wrong setup	4 KiB sequential reads over a 64 MiB region, *no* `O_DIRECT`
Right setup	Identical workload *with* `O_DIRECT` (page cache bypassed)
Fixed	QD=1, 5 repetitions of 4 s; test file rewritten sequentially just before
Run	first run (`20261021-103201`); read-only apart from the re-fill, so not repeated
Script	`SSD_Performance_Profiling/scripts/run.sh pitfall`

Table 10: Same 4 KiB sequential-read workload, direct vs. buffered: medians of 5 repetitions with the IQR in brackets. Only the `O_DIRECT` flag differs.

Metric	`O_DIRECT` (correct)	buffered (page cache)
IOPS	70 920 [68 204, 71 049]	1 334 404 [1 334 295, 1 334 424]
MiB/s	277 [266, 278]	5 213 [5 212, 5 213]
Mean latency (µs)	14.03 [14.00, 14.59]	0.68 [0.68, 0.68]
p99 latency (µs)	18.1 [18.1, 18.2]	1.15 [1.15, 1.15]

10.1 Explanation

The buffered run reports 1 334 404 IOPS at 5 213 MiB/s and sub-microsecond latency, a 19× inflation over the correct 70 920 IOPS. None of it is a storage measurement: after the first pass the 64 MiB region lives in the page cache, so every later read is a `memcpy` from RAM. Omitting `O_DIRECT` is the default behaviour of `open()`, so an AI-generated benchmark falls into this readily. Two cheap detectors flag it in our data, and a third is expected to:

1. The throughput exceeds the 3756 MiB/s PCIe 3.0 ×4 link ceiling (Section 2), which no device result can do.
2. Sub-microsecond "storage" latency is physically impossible for NAND flash.
3. *Untested:* it should collapse to device speed once the region exceeds free RAM (63 GiB here).

The methodology avoids the trap by construction: `O_DIRECT` is the default for every real run, and `make test` asserts that buffered re-reads are more than 5× faster than direct ones. Related traps are a wrong effective queue depth, caught by L/QD (Table 7), and wrong-order preconditioning (Section 9.3).

11 Anomalies and Limitations

11.1 Unexpected results

- **The SLC cache size depends on drive state** (20.8 GiB here, 3.8 and 3.1 GiB in earlier runs), and after a full fill neither 60 s nor 20 min of idle time restored much of it (Section 9).

- **Random writes beat random reads until the cache fills** (254 872 vs. 107 086 IOPS at QD=16), and the write collapses inside the sweeps were drive state, not block size or queue depth (Sections 5 and 7).
- **Span did not matter beyond** 4 GiB, contrary to hypothesis (iv) (Section 9.2).
- **Deep queues did not hide fragmentation** (9.2× at QD=32; Section 9.3), and in the first run **recent writes** halved read IOPS and raised p99.9/p50 to 24× (Section 8).

11.2 Limitations

- **Run order still matters.** The 60 GiB fill comes first, so later write points had less cache headroom despite the idle periods (Sections 5 and 7); `pitfall` and `layout` come from the superseded first run.
- **Short-term steady state.** One 60 GiB fill and 153 s of random writes, not the full-drive SNIA procedure; the cache size was measured once per drive state.
- **Closed-loop load only** (an open-loop generator would show delay against offered load).
- **No `iostat/perf` data;** L/QD bounds device-queue occupancy from above.
- **SMART only once** (root needed), after the 2026-10-21 runs: zero thermal events, media errors and over-temperature time, 100 % spare, but 28 % of rated life used, 72.61 TB written, 20 028 power-on hours and 2105 uninspected error-log entries. Wear may affect the cache size and tails.
- **Files on ext4, not the raw device** (at most 60 GiB, about 13 % of the drive); one drive, firmware and incompressible pattern.
- **Write points** use byte caps and 3–5 repetitions (∼15 % drive-state uncertainty); FIO, not the primary engine, agrees within 2 % on reads.

12 AI-Use Note

Summarized from `AI_USE_LOG.md` in the code repository.

Tools used

Claude Code (Claude Opus 5.5) for coding, debugging, and analysing the data: the initial harness up to 2026-10-20, then the 2026-10-21 rework and the analysis of the final runs. GCC gcc (GCC) 16.2.1 20260810 built the code. The AI wrote the code. It also produced a preliminary draft of the report, which I discarded; I wrote this report myself and verified all AI output. I executed the timed runs in a terminal on the machine in Table 1, using the scripts Claude Code wrote (`run.sh all`, then `SUSTAIN_GIB=60 run.sh device_state`, and later `rerun_clean.sh lean` after `sudo fstrim -v /`). Claude Code ran no timed benchmark; every number comes from those runs.

What they helped generate

`blkbench.c` (the O_DIRECT `libaio` benchmark), the data-safety gate (`safety.sh`) and its tests (`validate.sh`), the experiment driver (`run.sh`) with its CSV reduction, and the FIO job files and converter.

What was independently modified, tested, or rejected

Concrete corrections:

1. *The wrong device was being tested.* The scratch directory defaulted to the in-repo `data/scratch/`, which lives on the 2 TB SATA *HDD*, so the first full dataset showed millisecond QD=1 latencies. The scratch path was made overridable (`ACS_SCRATCH`) and pointed at the NVMe SSD. The AI's claim that "the NVMe is read-only to us" was false, as a one-line write test showed. The device check in `env.txt` was itself buggy: it stripped trailing digits from `nvme0n1p3` and printed `rotational: ?`. It now resolves the partition to its parent disk.
2. An early version measured *submission* latency rather than *completion* latency; it now times each I/O to completion.
3. A convenience default left the page cache enabled and reported ∼2 million IOPS. That case became the required pitfall (Section 10), and `O_DIRECT` became the default.
4. The report claimed a `0xA5` write fill, but the timed write buffers were never initialised. They are now filled once with an incompressible xorshift pattern, and `make test` checks that it does not compress.
5. The report claimed `validate.sh` refused `/dev/nvme0n1` when it only tested `/dev/sda`; it now tests both. It also claimed a direct-vs-buffered assertion that did not exist; the assertion has been added.
6. An AI-drafted sentence said p99 "rises monotonically with write fraction". The table contradicts it (pure write is below 50/50), and the sentence was corrected.
7. The first version of the sustained-write summary defined "burst" as the median of the first 10 s. On the real data, where the burst lasted only 3 s, that reported a misleading 66 MiB/s burst and a spurious cliff at 11 GiB. The definitions were fixed and the summary was regenerated from the unchanged raw log (`run.sh reprocess`).
8. The earlier report attributed the small (3.1 GiB) cache to the run order. The lean re-run on a trimmed, idle drive found 3.8 GiB, so that explanation was withdrawn (Section 9). The summary column `rand_steady_iops` is a byte-weighted phase average, and in the lean run a final 0.7 s at SLC speed doubles it. Checked against the per-second log, the text now quotes the per-second median of the eight full seconds as the steady value.
9. Claims carried over from the pilot data were re-checked against the final run and corrected where the data disagreed: the QD 1→4 scaling, the 26 % gain from QD 32 to 64 (15 %), the constant p99.9/p50 ratio, and the prediction that deep queues hide fragmentation.

Validation

`make test` checks the safety gate (both block devices, a symlink to one, `/etc/passwd`, `/`), the incompressible fill, physical orderings, the $L \leq \mathrm{QD}$ invariant, page-cache bypass and the FIO converter. FIO cross-checks `blkbench` (Section 3.1). Every figure traces to a committed raw/processed file (Section A).

13 Conclusion

At QD=1 the Micron 2210's write buffer makes 4 KiB random writes faster than reads. Block size moves the bottleneck from per-request latency to bandwidth, but even at QD=32 sequential reads reach only 56 % of the PCIe link, so the NAND back end is the ceiling, and writes saturate the ∼1.05 GiB/s SLC ingest rate even at QD=1. Mixing reads and writes is worse than either pure workload. The read knee lies at QD 64 under an explicit 10 %-per-doubling rule; beyond it latency

doubles per doubling of QD, and L/QD of 0.987–0.999 confirms that the saturation is the device's, not the harness's.

Device state matters as much as configuration. On a freshly trimmed, idle drive the SLC cache held 20.8 GiB, against 3.8 and 3.1 GiB in earlier runs, plausibly because it scales with free blocks; past it writes fell 17× and did not recover after a minute of idle time. The same exhaustion briefly cut writes to the QLC rate in the block-size and queue-depth sweeps, and it made the write knee rule return a meaningless QD=2. Reads did not slow with span beyond 4 GiB, but the write history of the data made "sequential" reads 6–9× slower, and in our first run recent heavy writes stretched p99.9/p50 from 6.4× to 24×. Temperature stayed far below the warning threshold.

The sharpest lessons are methodological. Without `O_DIRECT` the same workload reported page-cache fiction, and the first dataset was taken on the HDD until the `rotational` flag gave it away. A storage measurement can be trusted only if it bypasses the cache, verifies its effective queue depth, controls the data's write history and the drive's recent workload, and confirms which device it is talking to.

References

[1] J. Axboe, *Fio: Flexible I/O tester*, Version 3.42 on the test host; used to cross-check the libaio benchmark (blkbench). [Online]. Available: `https://github.com/axboe/fio`

[2] J. D. C. Little, "A proof for the queuing formula: $L = \lambda W$," *Operations Research*, vol. 9, no. 3, pp. 383–387, 1961. DOI: `10.1287/opre.9.3.383`

[3] Micron Technology, Inc., *Micron 2210 QLC SSD with NVMe: Product brief*, Rev. A, Micron Technology, Apr. 2020. [Online]. Available: `https://micron.cn/content/dam/micron/global/public/products/product-flyer/2210-qlc-ssd-product-brief.pdf`

[4] PCI-SIG, *PCI Express base specification, revision 3.0*, PCI-SIG, Nov. 2010.

[5] Storage Networking Industry Association, *Solid state storage (SSS) performance test specification (PTS)*, version 2.0.2, SNIA, Oct. 2020. [Online]. Available: `https://www.snia.org/tech_activities/standards/curr_standards/pts`

[6] B. Tallis, *The Intel SSD 660p SSD review: QLC NAND arrives for consumer SSDs*, AnandTech, Aug. 7, 2018. [Online]. Available: `https://www.anandtech.com/show/13078/the-intel-ssd-660p-ssd-review-qlc-nand-arrives`

A Reproducibility

Code and data live in this project's `code/` directory, the report in `latex/` (shared files in `common/`), at commit `65219fa68aef`.

```
gh repo clone SKempiak/ACS-Code ACS && cd ACS
git checkout 65219fa68aef
make tools                                  # common/code/build/tools/summarize
make -C SSD_Performance_Profiling/code      # build/blkbench (O_DIRECT, libaio)
make -C SSD_Performance_Profiling/code test # safety + sanity checks (~70 MiB written, deleted)
common/code/scripts/collect_env.sh --tex > common/latex/system_info.tex
export ACS_SCRATCH="$HOME/acs_ssd_scratch"  # a directory on the SSD under test
SSD_Performance_Profiling/code/scripts/run.sh dryrun   # device, link, space; writes nothing
SSD_Performance_Profiling/code/scripts/run.sh pitfall  # ran 2026-10-21 in run.sh all
SSD_Performance_Profiling/code/scripts/run.sh layout   # ditto
sudo fstrim -v /                            # 42.8 GiB at ~01:41; idle >= 30 min
```

```
SSD_Performance_Profiling/code/scripts/rerun_clean.sh  # run 20261022-021116
common/code/scripts/sync_data.sh SSD_Performance_Profiling
make ssd
```

`rerun_clean.sh` (Section 2) first checks free space, drive idleness and recent-TRIM evidence, then runs the steps of Table 2 with their idle periods. Its console output is in `data/raw/rerun_clean_20261022-021116.log`, and the root SMART log (taken after the 2026-10-21 runs) under `data/raw/smart/`. The superseded runs (`20261021-103201`, `20261021-105439`, and the lean `20261021-220614` with its `_lean` directories and log) remain under `data/raw/`. Without `ACS_SCRATCH`, the in-repo `data/scratch/` (an HDD here) is profiled; the safety gate applies either way. Knobs (`REPS`, `RUNTIME`, `WRITE_CAP_MIB`, `SUSTAIN_GIB`, `ENGINE=fio`, ...) are documented in `run.sh`.

Table 11: Raw data behind each figure and table (paths under `data/raw/` of the SSD project).

Figure / table	Raw run directory
Table 5	`zero_queue/20261022-021116`
Figures 1 and 2	`block_size/20261022-021116`
Figure 3, Table 6	`rw_mix/20261022-021116`
Figure 4, Table 7	`queue_depth/20261022-021116`
Figure 5	`tail/20261022-021116` (first run: `tail/20261021-103201`)
Table 10	`pitfall/20261021-103201`
Table 9	`layout/20261021-103201`
Tables 3 and 4	`fio_check/20261022-021116` (FIO JSON under `fio/`)
Figure 6, Table 8	`sustained/20261022-021116` (earlier: `sustained/20261021-105439`, `sustained/20261021-220614`)
Figure 7	`working_set/20261022-021116` (lean run: `working_set/20261021-220614`)
Table 2	`rerun_clean_20261022-021116.log`
tmpfs L/QD (Section 7)	`validate_tmpfs/20261021-231433.txt` (`make test` output, scratch on `/dev/shm`)

Each raw directory holds `raw.csv` and `env.txt`; `tools/summarize` reduces them to the CSVs in `data/`, from which every table and quoted number is read at compile time.